Python OOP Part 4: File Handling & Serialization - Beginner Notes

1. Operator Overloading

What is Operator Overloading?
Operator overloading allows you to define custom behavior for operators like `+`, `-`, `*`, `==`, etc. in classes.

Common Magic Methods:

class Point:
def **init**(self, x, y):
self.x = x
self.y = y

```
# Overload + operator
def __add__(self, other):
    return Point(self.x + other.x, self.y + other.y)

# Overload == operator
def __eq__(self, other):
    return self.x == other.x and self.y == other.y

# String representation
def __str__(self):
    return f"Point({self.x}, {self.y})"
```

p1 = Point(1, 2)
p2 = Point(3, 4)
p3 = p1 + p2
print(p3) # Output: Point(4, 6)
print(p1 == p2) # Output: False

Common Magic Methods:

- `__add__`: `a + b`
- `__sub__`: `a - b`
- `__mul__`: `a * b`
- `__eq__`: `a == b`
- `__lt__`: `a < b`
- `__str__`: `str(a)`

---

2. Writing Multiple Lines

Write single line:
```
file = open("data.txt", "w")
file.write("Hello, World!")
file.close()
```

Write multiple lines:
```
file = open("data.txt", "w")
file.write("Line 1\n")
file.write("Line 2\n")
file.write("Line 3\n")
file.close()
```

Write using writelines():
```
lines = ["Hello\n", "World\n", "Python\n"]
file = open("data.txt", "w")
file.writelines(lines)
file.close()
```

---

3. Writing to a Text File (Existing File)

Important: Mode matters!

- 'w' mode: Overwrites entire file
- 'a' mode: Appends to end of file
- 'r+' mode: Read and write without overwriting

This overwrites the entire file

```
file = open("data.txt", "w")
file.write("New content - old content is lost")
file.close()
```

This adds to existing file WITHOUT overwriting

```
file = open("data.txt", "a")
file.write("This is appended")
file.close()
```

---

4. How open() Works

Syntax: `open(filename, mode, encoding)`

File Modes:

- `'r'`: Read only (default)
- `'w'`: Write (overwrites)
- `'a'`: Append (add to end)
- `'r+'`: Read and write
- `'w+'`: Write and read (overwrites)
- `'x'`: Create new file (error if exists)
- `'b'`: Binary mode (e.g., 'rb', 'wb')

Open for reading

file = open("data.txt", "r")

Open for writing

file = open("data.txt", "w")

Open for appending

file = open("data.txt", "a")

Always close when done

file.close()

Return Value: File object with methods like `read()`, `write()`, `close()`

---

5. Append Mode

Append mode (`'a'`) adds data to end of file without deleting existing content.

Create initial file

file = open("log.txt", "w")
file.write("Event 1\n")
file.close()

Append more data

```
file = open("log.txt", "a")
file.write("Event 2\n")
file.write("Event 3\n")
file.close()
```

Read to verify

```
file = open("log.txt", "r")
print(file.read())
```

Output:

Event 1

Event 2

Event 3

```
file.close()
```

---

6. Writing Many Files

Create and write to multiple files:

Write to multiple files

```
files_data = {
"file1.txt": "Content of file 1\n",
"file2.txt": "Content of file 2\n",
"file3.txt": "Content of file 3\n"
}

for filename, content in files_data.items():
file = open(filename, "w")
file.write(content)
file.close()
```

Verify all files created

```
import os
print(os.listdir()) # Shows all files in directory
```

---

7. Saving Files at Any Location

Specify full file path:

Windows path

```
file = open("C:/Users/Documents/data.txt", "w")
file.write("Hello")
file.close()
```

Linux/Mac path

```
file = open("/home/user/documents/data.txt", "w")
file.write("Hello")
file.close()
```

Using pathlib (modern approach)

```
from pathlib import Path

path = Path("/home/user/documents/data.txt")
path.write_text("Hello")
```

Create directory if not exists

```
from pathlib import Path
path = Path("new_folder/data.txt")
path.parent.mkdir(parents=True, exist_ok=True)
path.write_text("Content")
```

---

8. Reading from Files - read()

read() reads entire file content as single string.

Create sample file

```
file = open("data.txt", "w")
file.write("Hello World\nPython Programming\nFile Handling")
file.close()
```

Read entire file

```
file = open("data.txt", "r")
content = file.read()
print(content)
```

Output:

Hello World

Python Programming

File Handling

```
file.close()
```

Read specific number of characters

```
file = open("data.txt", "r")
first_5 = file.read(5) # Read first 5 characters
print(first_5) # Output: Hello
file.close()
```

---

9. Reading from Files - readlines()

readlines() reads file into list of lines (each line as string with \n).

```
file = open("data.txt", "r")
lines = file.readlines()
print(lines)
```

Output: ['Hello World\n', 'Python Programming\n', 'File Handling']

Loop through lines

```
for line in lines:
print(line) # Each line includes \n at end
file.close()
```

Remove newline characters

file = open("data.txt", "r")
lines = file.readlines()
clean_lines = [line.strip() for line in lines]
print(clean_lines)

Output: ['Hello World', 'Python Programming', 'File Handling']

file.close()

---

10. Reading Big File Line by Line

Use readline() for memory-efficient reading of large files:

file = open("large_file.txt", "r")

Read one line at a time

while True:
line = file.readline()
if not line: # Empty string means end of file
break
print(line.strip()) # Strip newline

file.close()

Alternative: Loop directly on file object

file = open("large_file.txt", "r")
for line in file:
print(line.strip())
file.close()

Most memory efficient

with open("large_file.txt", "r") as file:
for line in file:
print(line.strip())

---

11. Using Context Manager With

`with` statement automatically opens and closes file (best practice).

Without 'with' (manual close needed)

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close() # Must remember to close
```

With 'with' statement (automatic close)

```
with open("data.txt", "r") as file:
content = file.read()
print(content)
```

File automatically closed here

Write with 'with'

```
with open("data.txt", "w") as file:
file.write("Hello World")
```

Multiple files

```
with open("input.txt", "r") as inp, open("output.txt", "w") as out:
content = inp.read()
out.write(content.upper())
```

Advantages:

- Automatic file closing
- Handles exceptions properly
- Cleaner, more readable code

---

12. Reading Big File in Chunks

Read large file in fixed-size chunks to save memory:

```
def read_in_chunks(filename, chunk_size=1024):
with open(filename, "rb") as file: # Binary mode
while True:
chunk = file.read(chunk_size)
if not chunk:
break
print(len(chunk), "bytes read")
```

read_in_chunks("large_file.txt", chunk_size=512)

Process chunks immediately (don't store all)

with open("large_file.txt", "r") as file:
while True:
chunk = file.read(8192) # Read 8KB at a time
if not chunk:
break
# Process chunk here
process_data(chunk)

---

13. Seek and Tell

seek() moves file pointer, tell() returns current position.

with open("data.txt", "r") as file:
tell() returns current position (0 at start)
print(file.tell()) # Output: 0

```
# Read 5 characters
file.read(5)
print(file.tell())  # Output: 5

# Seek to beginning
file.seek(0)
print(file.tell())  # Output: 0

# Read all from beginning
content = file.read()
print(content)
```

Seek to specific position

with open("data.txt", "rb") as file:
file.seek(10) # Move to byte 10
data = file.read(5) # Read 5 bytes from position 10

```
file.seek(-5, 2)  # Go to 5 bytes before end (2 = end of file)
last_data = file.read()
```

---

14. Problems with Text File and Working with Binary Files

Text File Problems:

- Line endings differ (Windows: \r\n, Linux: \n)
- Encoding issues (UTF-8, ASCII)
- Not suitable for binary data (images, executables)

Binary Files:

Write binary data

```
with open("image.bin", "wb") as file:
data = b'\x00\x01\x02\x03'
file.write(data)
```

Read binary data

```
with open("image.bin", "rb") as file:
data = file.read()
print(data) # Output: b'\x00\x01\x02\x03'
```

Work with actual image

```
with open("photo.jpg", "rb") as file:
image_data = file.read()
# Process binary image data
```

---

15. Working with Other Data Types in File Handling

Files store text, so convert data types:

Store list as text

```
numbers = [1, 2, 3, 4, 5]
with open("numbers.txt", "w") as file:
file.write(str(numbers)) # Convert to string
```

Read back and convert

```
with open("numbers.txt", "r") as file:
content = file.read()
numbers = eval(content) # Convert string back to list
print(numbers) # Output: [1, 2, 3, 4, 5]
```

Better approach: use JSON (next section)

---

16. Serialization & Deserialization

Serialization: Convert Python object to file/string format
Deserialization: Convert file/string format back to Python object

Without serialization (loses type)

```
data = {"name": "Alice", "age": 25}
with open("data.txt", "w") as file:
file.write(str(data))

with open("data.txt", "r") as file:
loaded = eval(file.read()) # Dangerous!
print(loaded) # Still a dict but unsafe
```

With proper serialization (JSON or Pickle - next sections)

---

17. Serialization using json Module - dump()

json.dump() writes Python object to file in JSON format.

```
import json
```

Python object

```
student = {
"name": "Alice",
"age": 20,
"grades": [85, 90, 88]
}
```

Serialize to file

```
with open("student.json", "w") as file:
json.dump(student, file) # Write as JSON
```

File now contains: {"name": "Alice", "age": 20, "grades": [85, 90, 88]}

Pretty print (formatted)

with open("student.json", "w") as file:
json.dump(student, file, indent=4)

json.dumps() - serialize to string (not file):
import json

data = {"name": "Bob", "age": 22}
json_string = json.dumps(data)
print(json_string) # Output: {"name": "Bob", "age": 22}
print(type(json_string)) # Output: <class 'str'>

---

18. Deserialization using json Module - load()

json.load() reads JSON file and converts to Python object.

import json

Read from JSON file

with open("student.json", "r") as file:
student = json.load(file) # Read and convert

print(student) # Output: {'name': 'Alice', 'age': 20, 'grades': [85, 90, 88]}
print(type(student)) # Output: <class 'dict'>

Access data

print(student["name"]) # Output: Alice
print(student["grades"][0]) # Output: 85

json.loads() - deserialize from string (not file):
import json

json_string = '{"name": "Bob", "age": 22}'
data = json.loads(json_string)
print(data) # Output: {'name': 'Bob', 'age': 22}
print(type(data)) # Output: <class 'dict'>

---

19. Serialization & Deserialization of Tuple

JSON converts tuples to lists (limitation).

import json

Tuple to JSON

coordinates = (10, 20, 30)

with open("coords.json", "w") as file:
json.dump(coordinates, file)

Read back

with open("coords.json", "r") as file:
loaded = json.load(file)
print(loaded) # Output: [10, 20, 30] - converted to LIST!
print(type(loaded)) # Output: <class 'list'>

Workaround: Convert back to tuple

with open("coords.json", "r") as file:
loaded = tuple(json.load(file))
print(loaded) # Output: (10, 20, 30)
print(type(loaded)) # Output: <class 'tuple'>

Use Pickle for preserving exact type:
import pickle

coordinates = (10, 20, 30)

with open("coords.pkl", "wb") as file:
pickle.dump(coordinates, file)

with open("coords.pkl", "rb") as file:
loaded = pickle.load(file)
print(loaded) # Output: (10, 20, 30)
print(type(loaded)) # Output: <class 'tuple'>

---

20. Serialization & Deserialization of Nested Dict

JSON handles nested dictionaries well:

import json

Nested dictionary

```
company = {
"name": "Tech Corp",
"employees": [
{"name": "Alice", "age": 30, "dept": "IT"},
{"name": "Bob", "age": 28, "dept": "HR"}
],
"location": {
"city": "Mumbai",
"country": "India"
}
}
```

Serialize

```
with open("company.json", "w") as file:
json.dump(company, file, indent=4)
```

Deserialize

```
with open("company.json", "r") as file:
loaded = json.load(file)
print(loaded["name"]) # Output: Tech Corp
print(loaded["employees"][0]["name"]) # Output: Alice
print(loaded["location"]["city"]) # Output: Mumbai
```

---

21. Serialization & Deserialization of Nested Objects

JSON doesn't serialize custom objects directly. Use `__dict__`:

```
import json

class Student:
def init(self, name, age):
self.name = name
self.age = age

student = Student("Alice", 20)
```

Serialize - convert object to dict using dict

```
with open("student.json", "w") as file:
json.dump(student.dict, file)
```

Deserialize

with open("student.json", "r") as file:
data = json.load(file)
loaded_student = Student(**data) # Unpack dict as arguments
print(loaded_student.name) # Output: Alice

Nested objects:
import json

class Address:
def **init**(self, city, country):
self.city = city
self.country = country

class Person:
def **init**(self, name, address):
self.name = name
self.address = address

address = Address("Mumbai", "India")
person = Person("Alice", address)

Serialize nested objects

with open("person.json", "w") as file:
json.dump(person.**dict**, file, default=lambda o: o.**dict**, indent=4)

Deserialize (manual reconstruction needed)

with open("person.json", "r") as file:
data = json.load(file)
loaded_person = Person(data["name"], None)
loaded_person.address = Address(data["address"]["city"], data["address"]["country"])

---

22. Pickling

Pickle serializes Python objects preserving exact type.

import pickle

Data to pickle

```
data = {
"list": [1, 2, 3],
"tuple": (4, 5, 6),
"dict": {"a": 1, "b": 2}
}
```

Serialize to file

```
with open("data.pkl", "wb") as file:
pickle.dump(data, file)
```

Deserialize from file

```
with open("data.pkl", "rb") as file:
loaded = pickle.load(file)
print(loaded)
print(type(loaded["tuple"])) # Output: <class 'tuple'>
```

Pickle with custom objects:

```
import pickle

class Student:
def __init__(self, name, age):
self.name = name
self.age = age

student = Student("Alice", 20)
```

Pickle preserves object structure

```
with open("student.pkl", "wb") as file:
pickle.dump(student, file)

with open("student.pkl", "rb") as file:
loaded = pickle.load(file)
print(loaded.name) # Output: Alice
print(type(loaded)) # Output: <class '__main__.Student'>
```

Pickle strings (not files):

```
import pickle

data = [1, 2, 3]
```

Serialize to bytes

```
serialized = pickle.dumps(data)
print(serialized) # Binary representation
```

Deserialize from bytes

```
deserialized = pickle.loads(serialized)
print(deserialized) # Output: [1, 2, 3]
```

---

23. Pickling vs JSON

Feature	JSON	Pickle
Format	Text (human readable)	Binary (unreadable)
Security	Safe (explicit parsing)	Unsafe (executes code)
Tuples	Converted to lists	Preserved as tuples
Custom Objects	Requires workaround	Native support
Language Support	Cross-language	Python only
File Size	Larger	Smaller
Speed	Slower	Faster

Table 1: Comparison of JSON and Pickle

JSON Example:

```
import json

data = {"name": "Alice", "skills": ("Python", "Java")}
with open("data.json", "w") as f:
json.dump(data, f)

with open("data.json", "r") as f:
loaded = json.load(f)
print(type(loaded["skills"])) # <class 'list'> - tuple lost!
```

Pickle Example:

```
import pickle

data = {"name": "Alice", "skills": ("Python", "Java")}
with open("data.pkl", "wb") as f:
pickle.dump(data, f)

with open("data.pkl", "rb") as f:
loaded = pickle.load(f)
print(type(loaded["skills"])) # <class 'tuple'> - preserved!
```

When to use:

- **JSON:** Web APIs, cross-language, public data
- **Pickle:** Python projects, complete object preservation, speed

---

Quick Reference

Operation	Code
Open file read	`with open("file.txt", "r") as f:`
Open file write	`with open("file.txt", "w") as f:`
Open file append	`with open("file.txt", "a") as f:`
Read all	`content = f.read()`
Read lines	`lines = f.readlines()`
Write	`f.write("text")`
JSON dump	`json.dump(obj, f)`
JSON load	`obj = json.load(f)`
Pickle dump	`pickle.dump(obj, f)`
Pickle load	`obj = pickle.load(f)`
Seek position	`f.seek(position)`
Tell position	`f.tell()`

Table 2: File Operations Quick Reference